LangGraph核心概念

- 来源: https://class.imooc.com/lesson/5284#mid=105035
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第9周 LangGraph：工作流编排
- 模块: 为什么需要LangGraph
- 课节: 图文 1-2 LangGraph核心概念

1-2 LangGraph核心概念

LangGraph核心概念

> [!TIP] 阅读指南 本节是对 LangGraph 核心概念的「地图说明书」。如果你已经大致理解了为什么需要 LangGraph，但还搞不清它到底由哪些部分组成，可以先快速通读本节，再在实战章节反复回来对照。 接下来，我们会正式进入四大核心概念：State（状态）、Node（节点）、Edge（边） 和 Graph（图）。

LangGraph的工作流由四个核心部分组成:

```
State(状态)
├─ 工作流的"数据容器"
└─ 记录所有中间结果

Node(节点)
├─ 工作流的"处理单元"
└─ 读取State→处理→更新State

Edge(边)
├─ 节点之间的"连接线"
└─ 固定边 + 条件边

Graph(图)
├─ 整个工作流的"容器"
└─ 组合State + Node + Edge

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

预览
```

让我们逐个深入理解。

State：工作流的数据中心

什么是State

State 是一个结构化的数据容器，存储工作流执行过程中的所有信息。

用写文章的例子类比:

- State就像一个文件夹
- 记录了搜集的资料、生成的大纲、写好的草稿、润色后的终稿
- 每个节点(搜资料、写大纲、写草稿。...)都会往文件夹里添加新内容

如何定义State

在 LangGraph 中，我们通常使用 Python 的 TypedDict 来定义 State 的结构：

```
from typing import TypedDict

class ArticleState(TypedDict):
    """文章写作的状态"""
    topic: str      # 主题
    outline: str    # 大纲
    draft: str      # 草稿
    final: str      # 终稿
```

State的特点

累积性：State 的字段在执行过程中会不断被填充。执行完一个步骤后，返回的数据会自动合并到 State 中，已有的数据会一直保留。

```
# 初始状态
{"topic": "Python异步编程"}

# 第1步后
{"topic": "Python异步编程", "outline": "1.基础 2.实战..."}

# 第2步后
{"topic": "Python异步编程", "outline": "1.基础...", "draft": "第一章..."}

# 所有中间结果都保留
```

Node：工作流的处理单元

什么是Node

Node 是一个普通的 Python 函数，负责处理 State 中的数据。

它的工作逻辑非常清晰，是一个典型的「输入-处理-输出」过程：

- 输入：Node 接收当前的 State 作为参数。

输入

- 处理：函数内部根据需要从 State 中提取数据，进行计算（如调用模型或工具）。
- 输出：计算完成后，返回一个字典，告诉系统需要更新哪些字段。

输出

你可以把它想象成一个流水线上的工人：从传送带上拿走零件（读取 State），组装好后（处理数据），再把成品放回传送带（返回字典更新 State）。

Node函数的标准格式

```
def node_function(state: ArticleState) -> dict:
    """
    输入:完整的State对象
    处理:执行某个具体任务
    输出:返回需要更新的字段(字典)
    """
    # 从State读取需要的数据
    topic = state["topic"]

    # 执行处理逻辑
    result = do_something(topic)

    # 返回要更新的字段
    return {"outline": result}
```

关键要点:

关键要点

- 输入参数:必须是State类型
- 返回值:字典，只包含需要更新的字段
- LangGraph会自动将返回的字典合并到State中

示例：三个Node

```
def generate_outline(state: ArticleState):
    """生成大纲节点"""
    topic = state["topic"]
    outline = f"关于{topic}的大纲:\n1.基础\n2.实战\n3.总结"
    return {"outline": outline}

def write_draft(state: ArticleState):
    """写草稿节点"""
    outline = state["outline"]
    draft = f"根据大纲:\n{outline}\n\n展开内容..."
    return {"draft": draft}

def polish(state: ArticleState):
    """润色节点"""
    draft = state["draft"]
    final = f"润色后:\n{draft}\n\n优化表达..."
    return {"final": final}
```

Edge：节点之间的连接

什么是Edge

Edge定义了节点之间的执行顺序。

用一个旅行的例子，最容易理解:

- Edge就是景点之间的路线
- 固定边:A景点后一定去B景点
- 条件边:根据天气决定去B还是C

固定边（add_edge）

用于顺序执行，无条件跳转:

```
# 添加固定边
workflow.add_edge("生成大纲", "写草稿")
workflow.add_edge("写草稿", "润色")
```

执行流程:

```
生成大纲 → 写草稿 → 润色


1
```

条件边（add_conditional_edges）

根据条件选择下一步:

```
def route_next(state):
    """路由函数:决定下一步去哪"""
    if state["quality_score"] > 80:
        return "发布"
    else:
        return "修改"

# 添加条件边
workflow.add_conditional_edges(
    "质量检查",            # 从哪个节点开始
    route_next,           # 路由函数
    {                     # 路由映射
        "发布": "publish_node",
        "修改": "revise_node"
    }
)
```

```
质量检查 ─┬─ 分数>80 → 发布
          └─ 分数≤80 → 修改


1
2
```

Graph：组装完整工作流

Graph是整个工作流的容器，把State、Node、Edge组合起来。

旅行例子:

- Graph就是完整的旅行计划书
- 包含所有景点(Node)、路线(Edge)、记录本(State)

```
from langgraph.graph import StateGraph, START, END

# 1. 初始化状态图：必须指定 State 的结构
workflow = StateGraph(ArticleState)

# 2. 注册节点：name 为唯一标识，func 为处理函数
workflow.add_node("生成大纲", generate_outline)
workflow.add_node("写草稿", write_draft)
workflow.add_node("润色", polish)

# 3. 编排路径：使用 START 和 END 常量定义边界
workflow.add_edge(START, "生成大纲")      # 指定入口点
workflow.add_edge("生成大纲", "写草稿")  # 添加从 start 到 end 的固定连接
workflow.add_edge("写草稿", "润色")
workflow.add_edge("润色", END)             # 指定终点

# 4. 编译应用：将逻辑图转换为可执行的 Runnable 对象
app = workflow.compile()

# 5. 调用执行：传入初始数据启动工作流
result = app.invoke({"topic": "Python异步编程"})
```

实战：最简工作流

现在让我们完整实现一个两节点的最简工作流，巩固上面的概念。

场景设计

任务:给定一个数字，计算它的平方和立方

流程:

流程

```
输入数字 → [计算平方] → [计算立方] → 输出结果

1
```

核心代码实现

> 完整源码文件: samples/chapter9/simple_langgraph_demo/simple_math_workflow.py

在 LangGraph 中构建一个最简工作流（如本例中的数学计算）主要分为四个核心步骤：

- 定义状态（State）：使用 TypedDict 规定数据流转的结构。

定义状态（State）

- 定义节点（Node）：编写具体的处理函数（如计算平方、立方）。

定义节点（Node）

- 构建图（Graph）：通过 StateGraph 将节点与边（START / END）连接。
- 编译与运行：调用 compile() 生成可执行对象并传入初始数据。

编译与运行

为了保持正文简洁，建议你直接打开配套的源码文件查看详细实现。代码中包含了详尽的内联注释，展示了状态是如何从输入开始，经过各个节点的计算，最终汇聚成完整结果的。

执行结果

```
计算平方:5² = 25
计算立方:5³ = 125

最终状态:
{'number': 5, 'square': 25, 'cube': 125}
```

State更新机制详解

这是很多初学者容易混淆的地方，让我详细说明。

```
# 初始State
{"number": 5}

# 执行"平方"节点后
# 节点返回:{"square": 25}
# LangGraph自动合并 →
{"number": 5, "square": 25}

# 执行"立方"节点后
# 节点返回:{"cube": 125}
# LangGraph自动合并 →
{"number": 5, "square": 25, "cube": 125}
```

规则：

- 节点只需返回新增或修改的字段

新增或修改

- LangGraph自动将返回值合并到State
- 已有字段会保留(累积性)

■ 学点英语

中文	English	音标	说明
状态	State	/steɪt/	工作流中结构化的数据容器，存储所有中间结果
节点	Node	/noʊd/	工作流的处理单元，接收 State 返回更新字典
边	Edge	/edʒ/	节点间的连接线，分固定边和条件边
条件路由	Conditional Routing	/kənˈdɪʃənəl ˈruːtɪŋ/	根据当前 State 动态选择下一个执行节点

下节预告

下一节，我们将用这些概念构建一个真实的文章写作 Agent，体验 LangGraph 在实战中的强大能力。